

TP3 - EDN

Artur Fonseca Costa

O código completo pode ser encontrado aqui.

2.1. A série de senos de $u(x, t)$ é dada por

$$u(x, t) = \sum_{n=1}^{\infty} b_n e^{-n^2 \pi^2 t} \sin(n\pi x), \text{ onde } b_n = 2 \int_0^1 f(x) \sin(n\pi x) dx.$$

Como a função $f(x)$ é definida por partes, tem-se

$$\begin{aligned} b_n &= 2 \left(\int_0^{1/2} 2x \sin(n\pi x) dx + \int_{1/2}^1 (2-2x) \sin(n\pi x) dx \right) \\ &= 4 \int_0^{1/2} x \sin(n\pi x) dx + 4 \int_{1/2}^1 \sin(n\pi x) dx - 4 \int_{1/2}^1 x \sin(n\pi x) dx \\ &= 4 \left[-\frac{x}{n\pi} \cos(n\pi x) + \frac{1}{n^2 \pi^2} \sin(n\pi x) \right] \Big|_0^{1/2} + 4 \left[\frac{x-1}{n\pi} \cos(n\pi x) - \frac{1}{n^2 \pi^2} \sin(n\pi x) \right] \Big|_{1/2}^1 \\ &= \frac{8}{n^2 \pi^2} \sin\left(\frac{n\pi}{2}\right). \end{aligned}$$

Nota-se que os coeficientes pares são nulos. Então, a solução é dada por

$$u(x, t) = \frac{8}{\pi^2} \sum_{n=0}^{\infty} \frac{(-1)^n}{(2k+1)^2} e^{-(2k+1)^2 \pi^2 t} \sin((2n+1)\pi x),$$

e a soma parcial de k termos é

$$u_k(x, t) = \frac{8}{\pi^2} \sum_{n=0}^k \frac{(-1)^n}{(2k+1)^2} e^{-(2k+1)^2 \pi^2 t} \sin((2n+1)\pi x).$$

Sendo $|\sin(x)| \leq 1$ para todo $x \in \mathbb{R}$ e $|e^{-x}| \leq 1$ para todo $x \in [0, \infty)$, pode-se aproximar a solução com

$$|u - u_k| \leq \frac{8}{\pi^2} \sum_{n=k+1}^{\infty} \frac{1}{(2k+1)^2} \leq \frac{8}{\pi^2} \int_k^{\infty} \frac{1}{(2x+1)^2} dx = \frac{4}{\pi^2(2k+1)}.$$

O erro desejado é $\varepsilon \leq 0.01$, então basta que $|u - u_k| \leq 0.01$:

$$\frac{4}{\pi^2(2k+1)} \leq 0.01 \longrightarrow 2k+1 \geq \frac{4}{0.01\pi^2} \approx 40.5 \longrightarrow k \geq 20.$$

Logo, basta usar os primeiros 21 termos $\left(\sum_{n=0}^{20}\right)$ para obter um erro inferior a 0.01.

2.2. O método das linhas depende de uma escolha de aproximação para $\partial_t u(x, t)$. Foi escolhido o método de Euler, cuja aplicação ao método das linhas é o método das diferenças progressivas:

$$u_{m,j+1} = u_{m,j} + \mu(u_{m-1,j} - 2u_{m,j} + u_{m+1,j}),$$

onde $\mu = \frac{\Delta t}{(\Delta x)^2}$. Sua implementação pode ser vista abaixo:

```

void ftcs(double x_0, double X, double t_0, double T, double dx, double dt){
    const int M = (X - x_0) / dx;
    const int N = (T - t_0) / dt;
    const double mu {dt / (dx * dx)};

    for (int m {}; m <= M; m++)
        x[m] = x_0 + m * dx;

    for (int j {}; j <= N; j++)
        t[j] = t_0 + j * dt;

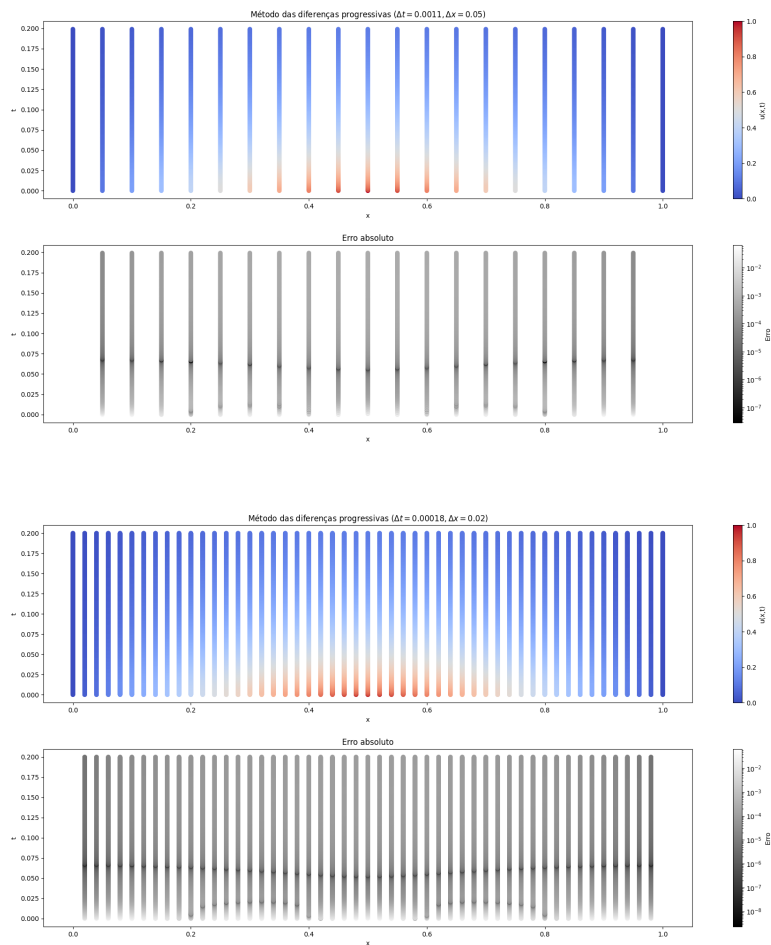
    for (int m {}; m <= M; m++)
        u[0][m] = f(x_0 + m * dx);

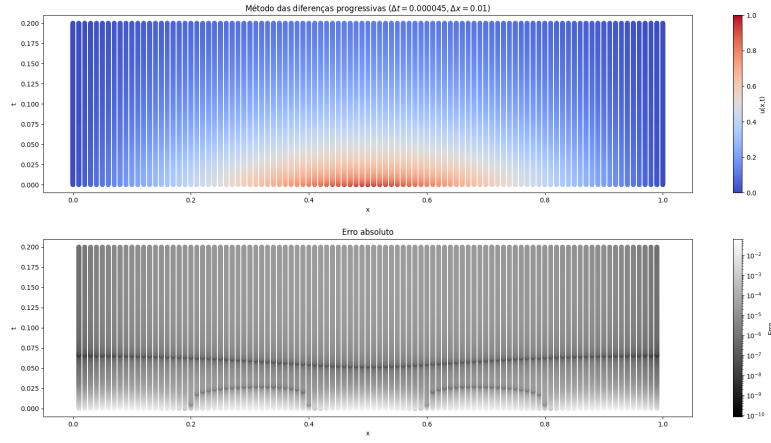
    for (int j {}; j <= N; j++){
        u[j][0] = 0;
        u[j][M] = 0;
    }

    for (int j {}; j < N; j++)
        for (int m {1}; m <= M - 1; m++)
            u[j + 1][m] = u[j][m] + mu * (u[j][m - 1] - 2 * u[j][m] + u[j][m + 1]);
}

```

Sabe-se que esse método é convergente se $\mu \leq \frac{1}{2}$, então os testes foram feitos com essa restrição em mente. Em casos onde essa restrição foi ignorada ($\Delta t = 0.0001$ e $\Delta x = 0.01$, por exemplo), o programa produziu valores inválidos (NaN) e não foi possível gerar o mapa de calor. Foram testados os valores $\Delta x = 0.01, 0.02, 0.05$:





Nota-se, em particular, que a solução chega a um estado próximo do estacionário muito rápido, visto que os mapas de calor mostram t no intervalo $[0, 0.02]$. O erro absoluto é calculado como $u(x_m, t_j) - u_{m,j}$, onde $u(x, t)$ é a aproximação pela série de Fourier do item (2.1) e $u_{m,j}$ é a aproximação gerada pelo método.

2.3. O método de diferenças progressivas foi utilizado no item (2.2), com a proporção $\mu = \frac{5}{11}$. Como comentado anteriormente, com $\mu = \frac{5}{9} > \frac{1}{2}$, os valores gerados pelo método explodiram rapidamente e não foi possível gerar um gráfico. Sabe-se que o mesmo não ocorre com o método de diferenças regressivas e o método de Crank-Nicolson:

$$u_{m,j+1} = u_{m,j} + \mu(u_{m-1,j+1} - 2u_{m,j+1} + u_{m+1,j+1});$$

$$u_{m,j+1} = u_{m,j} + \frac{\mu}{2}(u_{m-1,j} - 2u_{m,j} + u_{m+1,j} + u_{m-1,j+1} - 2u_{m,j+1} + u_{m+1,j+1}).$$

Por serem métodos implícitos, a implementação desses métodos envolve resolver um sistema linear a cada passo j :

```
void btcs(double x_0, double X, double t_0, double T, double dx, double dt){
    const int M = (X - x_0) / dx;
    const int N = (T - t_0) / dt;
    const double mu {dt / (dx * dx)};

    std::vector<double> x(M + 1);
    std::vector<double> t(N + 1);
    std::vector<std::vector<double>> u(N + 1, std::vector<double>(M + 1));

    for (int m {}; m <= M; m++){
        x[m] = x_0 + m * dx;

        for (int j {}; j <= N; j++){
            t[j] = t_0 + j * dt;

            for (int m {}; m <= M; m++){
                u[0][m] = f(x_0 + m * dx);

                for (int j {}; j <= N; j++){
                    u[j][0] = 0;
                    u[j][M] = 0;
                }

                std::vector<double> a(M - 1, -mu);
                std::vector<double> b(M - 1, 1 + 2 * mu);
                std::vector<double> c(M - 1, -mu);

                for (int j {}; j < N; j++){
                    a[0] = 0;
                    c[M - 2] = 0;

                    std::vector<double> d(M - 1);
```

```

        for (int m {1}; m < M; m++)
            d[m - 1] = u[j][m];

        std::vector<double> solution = solveTridiagonal(a, b, c, d);
        for (int m {1}; m < M; m++)
            u[j + 1][m] = solution[m - 1];
    }
}

void crankNicolson(double x_0, double X, double t_0, double T, double dx, double dt){
    const int M = (X - x_0) / dx;
    const int N = (T - t_0) / dt;
    const double mu {dt / (dx * dx)};

    std::vector<double> x(M + 1);
    std::vector<double> t(N + 1);
    std::vector<std::vector<double>> u(N + 1, std::vector<double>(M + 1));

    for (int m {}; m <= M; m++)
        x[m] = x_0 + m * dx;

    for (int j {}; j <= N; j++)
        t[j] = t_0 + j * dt;

    for (int m {}; m <= M; m++)
        u[0][m] = f(x_0 + m * dx);

    for (int j {}; j <= N; j++){
        u[j][0] = 0;
        u[j][M] = 0;
    }

    std::vector<double> a(M - 1, -mu);
    std::vector<double> b(M - 1, 1 + 2 * mu);
    std::vector<double> c(M - 1, -mu);

    for (int j {}; j < N; j++){
        a[0] = 0;
        c[M - 2] = 0;

        std::vector<double> d(M - 1);
        for (int m {1}; m < M; m++)
            d[m - 1] = u[j][m];

        std::vector<double> solution = solveTridiagonal(a, b, c, d);
        for (int m {1}; m < M; m++)
            u[j + 1][m] = solution[m - 1];
    }
}

```

As implementações são extremamente parecidas, utilizando a mesma resolução do sistema linear por meio de uma eliminação gaussiana específica para a matriz tridiagonal destes métodos. Assim como para o método de diferenças progressivas no item (2.2), foram testados os valores $\mu = \frac{5}{9}$ e $\mu = \frac{5}{11}$. Aqui, no entanto, é importante pontuar que os métodos são incondicionalmente estáveis, então não há nenhum problema de convergência.

2.4. Como mencionado no item (2.2), todos os gráficos já ilustram o erro $u(x_m, t_j) - u_{m,j}$, onde $u(x, t)$ é a série de Fourier, considerada aqui como a solução exata.

